실전 웹서비스개발 – 5주차

Database 기본 개념 및 MERN Stack 개발환경의 이해

학습 목표

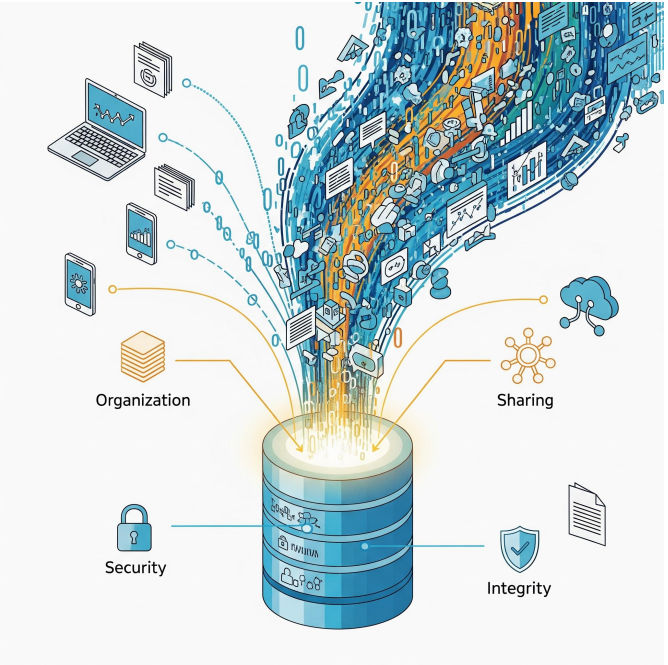
- Database 기본 개념
- Database 설계 방법
- MongoDB 기본 개념
- MERN Stack 종합

데이터베이스 기초

데이터베이스

공유되어 사용될 목적으로 통합 관리되는 데이터의 집합

- 여러 사용자가 공유하고 접근 가능
- 무결성 및 보안이 중요
- 이용자에게 필요한 정보를 제공하기 위한 서비스의 심장부



DBMS(Database Management System)

- 데이터베이스를 만들고, 저장 및 관리할 수 있는 기능들을 제공하는 응용 프로그램

데이터베이스 역사

1960s (시작: 네트워크/계층형, Charles Bachman)
- 기업 전산에서 데이터 독립성 요구가 커지며 DBMS 개념이 등장.

1970 (관계형 이론: E. F. Codd)
- 데이터를 엔티티와 관계로 표현
- 관계대수/관계해석 기반의 정형 질의(Query) 개념 정립.
- 정규화로 중복 최소화, 논리·물리적 데이터 독립성 강화.

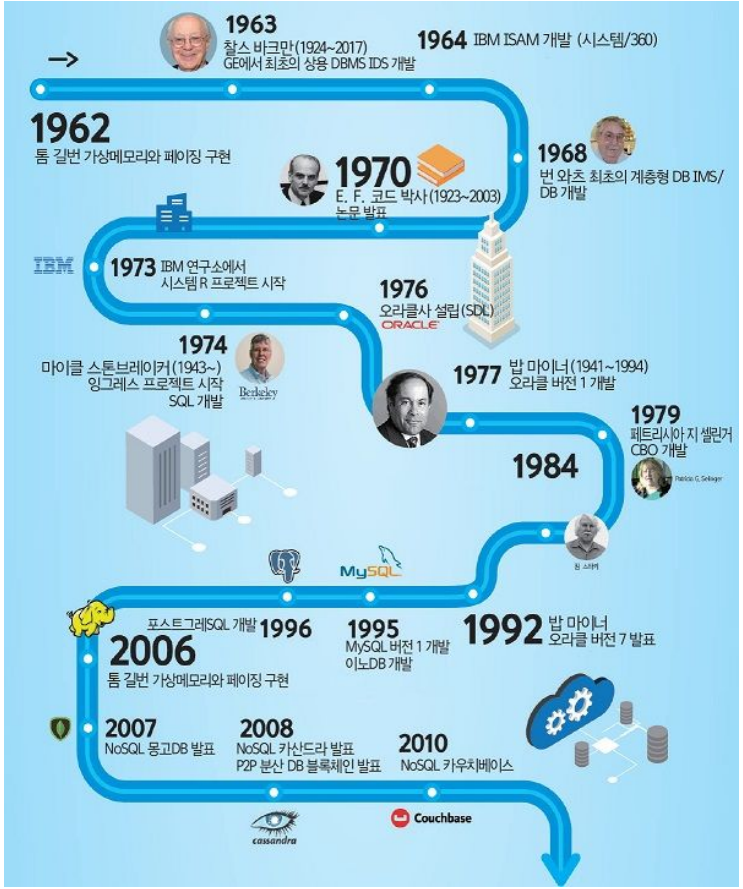
1970s (System R & 핵심 기술 완성)
- IBM System R에서 SQL 최초 구현
- RDBMS 필수 아키텍처가 표준화.

1970s~1990s (오픈·상용 생태 확장)
- ANSI SQL 표준화
- 클라이언트/서버 아키텍처 보급

1990s (Oracle 상용 선도: Larry Ellison/Bob Miner)
- 대규모 상용 워크로드에서 확장성·고가용성으로 RDBMS가 기업 표준으로 자리잡음.

1995~ (오픈소스 부상: MySQL + InnoDB, PostgreSQL)
- LAMP(Linux + Apache + MySQL + PHP) 스택과 함께 대중화

2000s (웹·빅데이터: NoSQL 등장)
- 웹 규모 확장성 요구로 NoSQL 확산
- RDBMS는 여전히 코어 시스템으로 사용



데이터베이스 유형

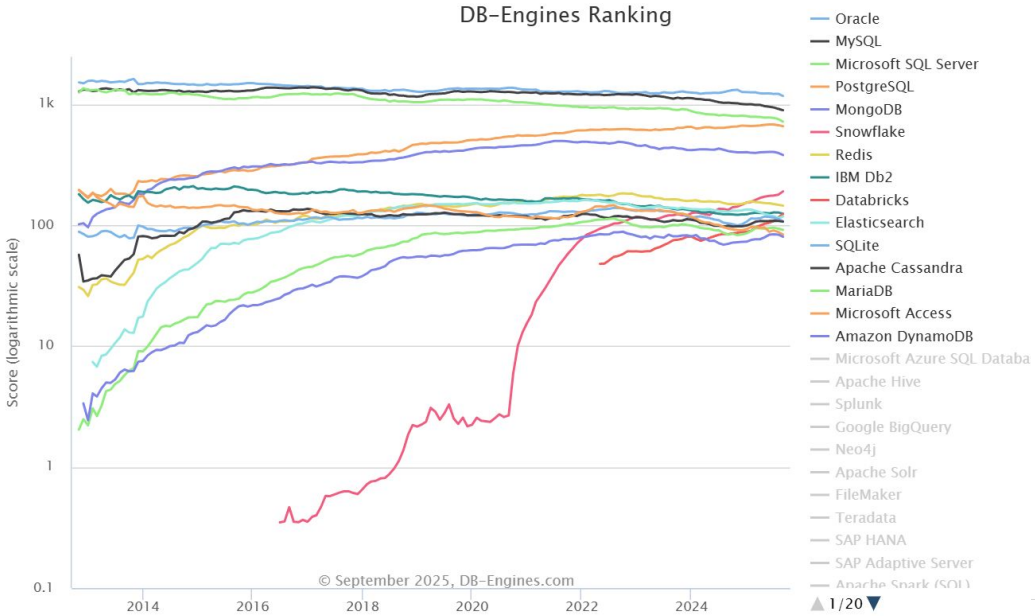
관계형 데이터베이스(RDBMS)

- 데이터를 테이블 형태로 구조화하여 저장하고, 각 테이블 간에 관계를 맺어 데이터를 관리
- 데이터의 정합성을 유지하고, SQL이라는 표준 쿼리 언어를 사용하여 복잡한 데이터 조작 및 검색이 가능
- 데이터의 구조 변경이 어렵고, 대규모의 비정형 데이터 처리에는 비효율적
- **주요 RDBMS** : Oracle, MySQL, PostgreSQL, MariaDB

비관계형 데이터베이스(NoSQL DBMS)

- 유연한 형태로 비정형 데이터를 저장
- 대규모의 분산 환경에서 뛰어난 확장성을 가지며, 유연한 데이터 모델링이 가능
- 복잡한 쿼리나 트랜잭션 처리가 어려울 수 있음
- **유형별**
  - **문서형(Document) DBMS**: JSON과 같은 문서 형태로 데이터를 저장, **MongoDB**
  - **키-값형(Key-Value) DBMS**: 키와 값의 형태로 데이터를 저장하여 빠른 검색이 가능, **Redis**
  - **그래프형(Graph) DBMS**: 노드와 엣지 형태로 복잡한 관계를 표현하고 저장

2025 데이터베이스 트렌드



Rank Sep 2025	Rank Aug 2025	Rank Sep 2024	DBMS	Database Model	Score Sep 2025	Score Aug 2025	Score Sep 2024
1.	1.	1.	Oracle	Relational, Multi-model	1170.62	-50.08	-115.97
2.	2.	2.	MySQL	Relational, Multi-model	891.77	-23.69	-137.72
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	717.32	-36.84	-90.45
4.	4.	4.	PostgreSQL	Relational, Multi-model	657.17	-14.08	+12.81
5.	5.	5.	MongoDB	Document, Multi-model	380.50	-15.08	-29.74
6.	6.	↑ 7.	Snowflake	Relational	190.19	+11.29	+56.47
7.	7.	↓ 6.	Redis	Key-value, Multi-model	145.17	-2.02	-4.25
8.	8.	↑ 9.	IBM Db2	Relational, Multi-model	124.19	-3.12	+1.14
9.	9.	↑ 14.	Databricks	Multi-model	124.06	+8.25	+39.82
10.	10.	↓ 8.	Elasticsearch	Multi-model	118.26	+3.99	-10.53
11.	11.	↓ 10.	SQLite	Relational	107.88	-4.72	+4.53
12.	12.	↓ 11.	Apache Cassandra	Wide column, Multi-model	106.98	-1.53	+8.04
13.	13.	↑ 15.	MariaDB	Relational, Multi-model	91.46	-2.13	+8.02
14.	14.	↓ 12.	Microsoft Access	Relational	83.61	-4.15	-10.15
15.	15.	↑ 17.	Amazon DynamoDB	Multi-model	80.28	-3.20	+10.22
16.	16.	16.	Microsoft Azure SQL Database	Relational, Multi-model	79.18	+3.34	+6.23
17.	17.	↑ 18.	Apache Hive	Relational	76.10	+5.06	+23.02
18.	18.	↓ 13.	Splunk	Search engine	75.77	+6.00	-17.26
19.	19.	19.	Google BigQuery	Relational	66.00	+0.82	+13.33
20.	20.	↑ 21.	Neo4j	Graph	53.78	-0.69	+11.10

https://db-engines.com/en/ranking

데이터베이스 관련 직군

데이터 모델러

- 데이터 구조, 관계, 제약 조건을 설계

데이터 아키텍트

- DBMS 선택 및 전체 데이터 아키텍처를 책임

데이터 엔지니어

- 데이터 파이프라인과 데이터 웨어하우스를 설계하고 구축

DBA(Database Administrator)

- 데이터베이스 운영, 모니터링, 백업, 보안 및 튜닝을 담당



데이터베이스 설계

데이터베이스 설계

서비스 요구 사항을 데이터 모델로 변환하여, 정확하고 빠르며 잘 확장되는 시스템을 만드는 작업

핵심 작업

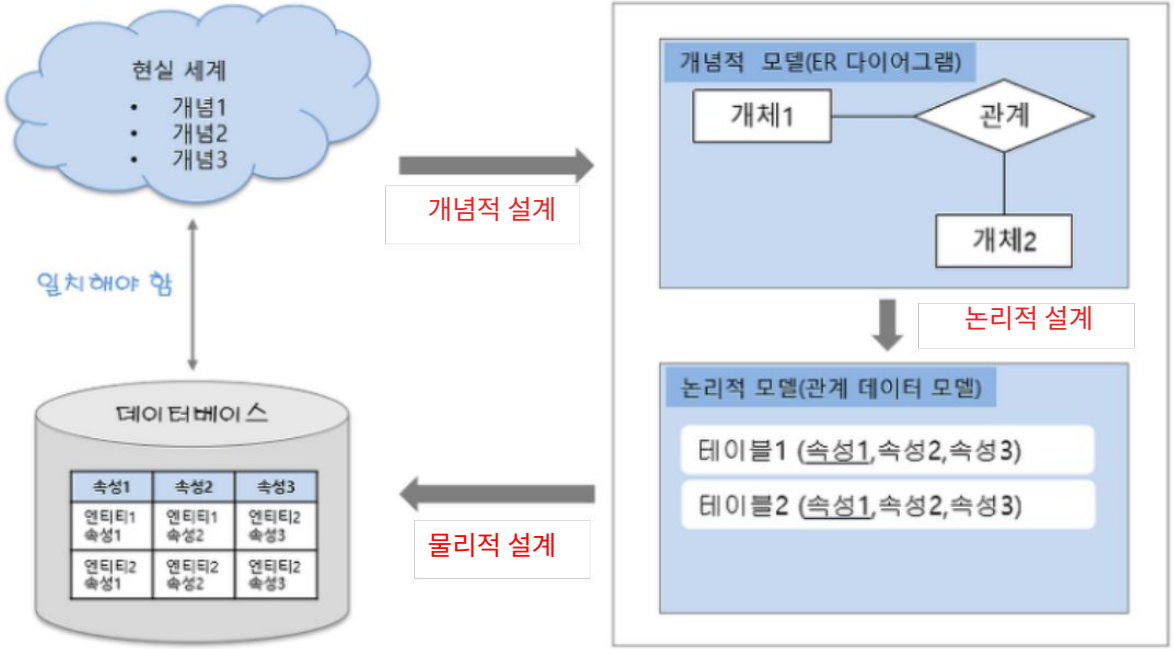
- 모델링
- 인덱스 설계
- 제약 설정
- 저장·분할 전략 수립
- 운영 계획 수립

왜 중요한가

- **정확성**: 잘못된/중복 데이터 방지
- **성능**: 적절한 인덱스·스키마로 응답속도/동시성 확보, 비용 절감
- **확장성**: 트래픽·데이터 증가에 맞춰 확장
- **개발속도**: 변경에 강한 스키마와 마이그레이션 전략을 세우면 빠른 개발 속도 유지

데이터베이스 설계 목표

구현하고자 하는 서비스를 구성하는 주요 개념들이 데이터베이스 구조에 일치하도록 설계



데이터베이스 설계 절차

개념적 설계

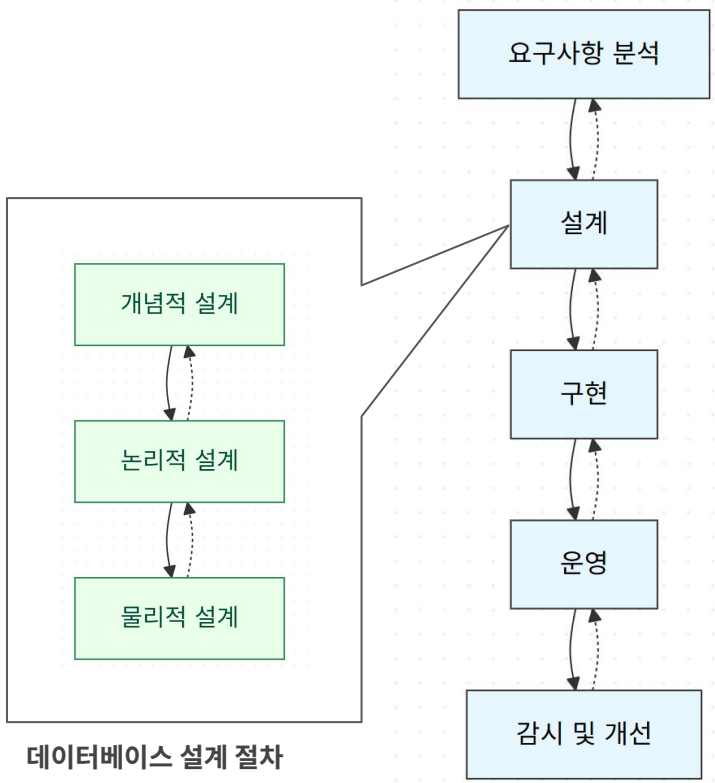
- 서비스의 요구를 데이터 세계의 개념(사물·행위)로 표현

논리적 설계

- DBMS가 처리 가능한 논리적 데이터 스키마 생성

물리적 설계

- 효율적이고 구현 가능한 물리적 데이터 구조 생성



데이터베이스 설계 절차

데이터베이스 개발 생명 주기

개념적 설계

서비스의 요구를 데이터 세계의 개념(사물·행위)로 표현

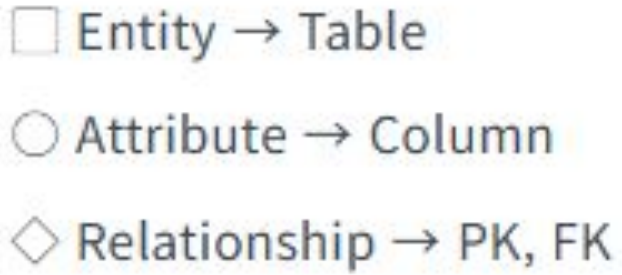
- **엔티티(Entity)** : 저장할 대상의 종류 (예: 사용자, 식당, 주문)
  - **약한 엔티티(Weak Entity)**: 다른 엔티티의 키 없이는 식별/존재가 어려운 엔티티(예: 주문의 주문항목)
- **속성(Attribute)** : 엔티티를 설명하는 값의 항목 (예: 이름, 전화번호)
  - **식별자/키(Identifier)**: 엔티티 인스턴스를 유일하게 구분하는 속성(예: user_id)
  - **도메인/값 범위(Domain)**: 속성이 가질 수 있는 합법적 값의 집합과 단위/형식(예: 평점 1~5 정수)
- **관계(Relationship)** : 엔티티 간의 연관성/연결(예: 사용자—리뷰 작성)
  - **카디널리티(Cardinality)**: 한쪽 엔티티가 다른 쪽과 맺을 수 있는 개수 규칙(1:1, 1:N, N:M)
  - **참여 제약(Participation)**: 관계에 반드시 참여해야 하는지(필수) 또는 해도 되고 안 해도 되는지(선택)

논리적 설계

DBMS가 곧바로 이해·변환할 수 있는 논리 데이터 ***스키마(Schema)***를 만드는 작업

스키마(Schema)

- 데이터 구조와 규칙의 설계서(엔티티, 속성, 키, 제약, 관계)



키(Key)

- **PK(Primary Key)**
  - 행/문서의 유일 식별자
  - 중복·NULL 금지
- **FK(Foreign Key)**
  - 다른 테이블의 PK 참조

물리적 설계

데이터베이스의 논리적 구조를 선택한 DBMS가 실제로 저장/조회할 수 있는 효율적이고 구현 가능한 물리 구조로 구체화

- 데이터베이스에 포함될 여러 파일 타입에 대한 저장 레코드의 양식, 순서, 접근 경로를 표현
- 데이터베이스 시스템의 성능에 중대한 영향을 줌

저장 레코드(Stored Record)

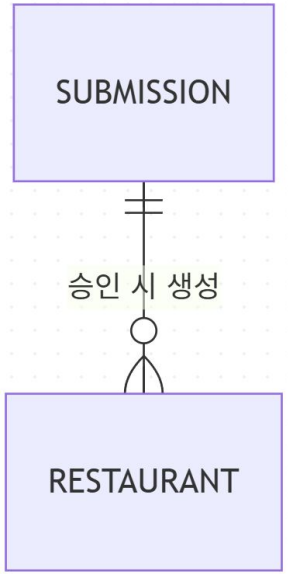
- 디스크(또는 SSD) 페이지(Page) 안에 저장되는 가장 기본적인 물리 단위의 “행/문서”

결정하는 내용

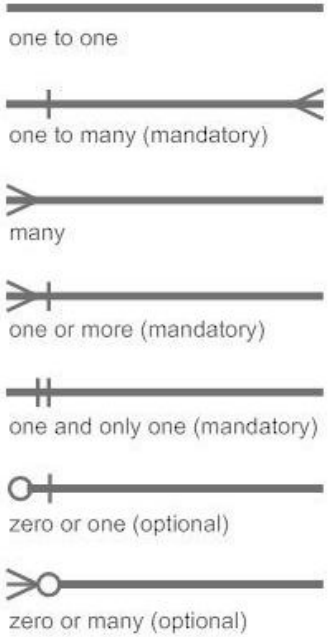
- 저장 레코드 형식
- 테이블 조직
- 인덱스 설계(접근 경로)
- 파일/스토리지 옵션
- 트랜잭션·동시성·복구
- 샤딩/복제(스케일링)

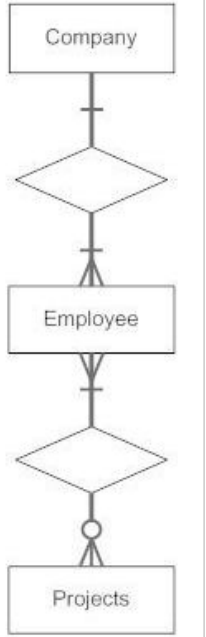
ERD(Entity–Relationship Diagram)

엔티티, 속성, 관계, 카디널리티를 그림으로 표현한 데이터 설계도



하나의 제보(SUBMISSION)는 승인될 때 0개 또는 1개의 맛집(RESTAURANT)을 생성할 수 있다





제약(Constraints) / 무결성(Integrity)

제약(Constraint)

- 잘못된/중복/모순 데이터를 저장 자체를 거부

무결성(Integrity)

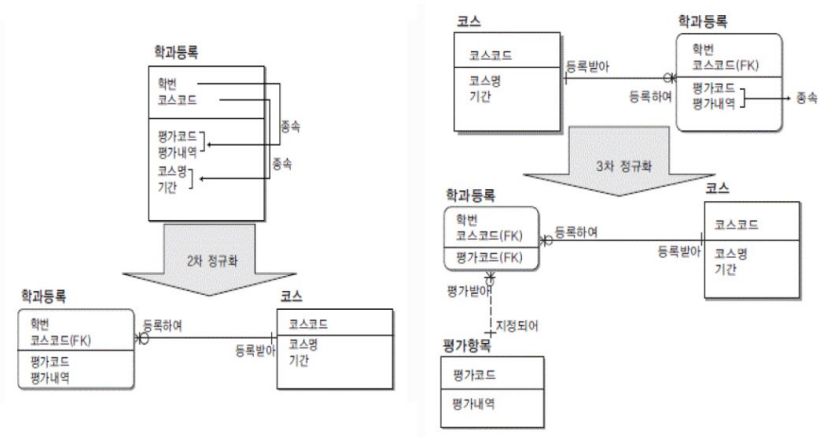
- 데이터가 항상 올바른 상태를 유지하는 성질
- 데이터 무결성은 제한이나 규칙에 의해 데이터베이스 시스템이 강제

무결성 유형

- **개체 무결성(Entity integrity)**
  - 모든 개체는 기본 키(primary key)를 가져야 하며 기본 키로 선택된 속성은 고유하여야 하며 빈 값은 허용치 않는다
- **참조 무결성(Referential integrity)**
  - FK는 참조 무결성을 유지해야 하며, NULL 허용 여부는 스키마(제약) 설정에 따라 결정된다.
- **도메인 무결성(Domain integrity)**
  - 값의 타입·범위·형식을 준수

정규화(Normalization)

중복된 데이터를 허용하지 않도록 테이블을 분해하여 설계



목표

- 무결성(Integrity)를 유지
- DB의 저장 용량 최적화

단계별

- **제1정규화(1NF)** : 다중값 분해 - 컬럼이 원자값(Atomic Value, 하나의 값)을 갖도록 테이블을 분해
- **제2정규화(2NF)** : 부분 종속 제거 - 제1 정규화를 진행한 테이블에 대해 완전 함수 종속을 만족하도록 테이블을 분해
- **제3정규화(3NF)** : 이행 종속 제거 - 제 2정규화를 진행한 테이블에 대해 이행적 종속을 없애도록 테이블을 분해
- **BCNF(Boyce-Codd Normal Form)** : 결정자ㄷ후보키 - 모든 결정자가 후보키 집합에 속하도록 테이블을 분해

학습 : https://en.wikipedia.org/wiki/Database_normalization

SQL(Structured Query Language)

관계형 데이터베이스(RDBMS)를 다루는 표준 언어

- 데이터 정의(DDL)·조작(DML)·권한 제어(DCL)·트랜잭션 제어(TCL)

대표 엔진

- Oracle
- MySQL
- MSSQL
- PostgreSQL
- MariaDB

```sql
-- INSERT (새 데이터)
INSERT INTO users(email, nickname) VALUES ('hyun@ajou.ac.kr', 'hyun');

-- UPDATE (부분 변경)
UPDATE restaurants SET rating = 4.5 WHERE id = 101;

-- DELETE (삭제)
DELETE FROM reviews WHERE id = 999;

-- SELECT (필터/정렬/제한)
SELECT id, name, rating
FROM restaurants
WHERE category = '라멘' AND rating >= 4.0
ORDER BY rating DESC, id ASC
LIMIT 20;
```

NoSQL(Not only SQL)

관계형(테이블·SQL) 대신 다양한 형식으로 데이터를 저장·조회하는 DB 계열의 총칭

특징

- 스키마 유연성(일관성과 확장성 사이의 절충)
  - 명시적인 스키마 없이 동작하고 데이터 형식을 유연하게 변경 가능
  - 암묵적인 스키마는 존재하지만 관계형 DB 처럼 강제하지 않음
- 수평 확장(Sharding) 쉬움
  - 샤딩, Sharding - 하나의 큰 데이터/트래픽을 여러 대의 서버(노드)로 나누어 처리

NoSQL 종류

- **Key-Value - Redis, DynamoDB**
  - 초고속 캐시/세션/카운터, 단순 조회에 강함
- **Document - MongoDB**
  - JSON 같은 문서 단위 저장, 임베딩 vs 참조 선택
  - 장점: 스키마 유연, 중첩 구조 한 번에 조회
- Column-family - HBase, Cassandra
  - 거대한 시계열/로그/메트릭에 최적
- Graph - Neo4j
  - 경로/연결 중심 질의(추천, 소셜)

MongoDB

MongoDB



문서 지향(Document-oriented) NoSQL DB

- mongo는 humongous를 줄인 표현. 즉 '매우 큰 DB'라는 뜻.
- 원래 GNU Affero General Public License(AGPL), 2018년 10월 이후 Server Side Public License(SSPL)로 변경
- BSON(Binary JSON)이라는 독자적인 형식을 사용, 데이터의 타입까지 포함되어 이진 형태로 인코딩되어 저장

활용

- **형태가 자주 바뀌는 데이터**: 프로필, 설정, 이벤트 로그
- **계층/중첩 구조가 자연스러운 데이터**: 주문→주문항목, 글→댓글
- **지리/텍스트 검색이 필요한 서비스**: 지도, 검색창
- **읽기 위주 대용량 스케일-아웃**: 리플리카셋/샤딩으로 수평 확장, 캐시처럼 읽기 분산
- **운영 데이터 실시간 집계**: 대시보드/피드 랭킹 등

MongoDB 핵심 개념

Document

- 기본적인 데이터(RDBMS의 row에 해당)

Collection

- Document의 모음(RDBMS의 Table에 해당)

DB

- Collection 모음(RDBMS와 동일)

BSON 타입

- String, Number, Boolean, Date, Array, Object, ObjectId, Decimal128

MongoDB Atlas

MongoDB 팀이 운영하는 완전관리형(Managed) 멀티클라우드 DB 플랫폼

- 배포/백업/모니터링/확장, 보안(네트워크·계정)까지 제공
- AWS/GCP/Azure 중 선택, 지역(Region)도 선택 가능

특징

- 설치 無: 로컬 설치 없이 곧바로 실습/배포
- 확장성: 필요 시 수직/수평 확장, 멀티 리전 구성 가능
- 운영 기능: 백업, 모니터링, 알람, 자동 복구
- 개발 생산성: Atlas Search, Vector Search, Data API, Triggers 등 원스톱



과금

- **Free(M0)**: 학습·PoC 용. 프로젝트 시작/수업용 기본 선택
- **Flex**: 사용량 기반(변동 부하에 적합)
- **Dedicated**: 예측 가능한 프로덕션 워크로드

MERN Stack 실습

MERN Stack

자바스크립트로 클라이언트, 서버, DB를 일관되게 개발

- **M**ongoDB: 문서형 NoSQL 데이터베이스
- **E**xpress: Node.js 웹 프레임워크
- **R**eact: 프론트엔드 UI 라이브러리
- **N**ode.js: 서버 런타임

활용

- 빠른 프로토타입/스타트업 MVP
- JSON 중심 API + 웹/모바일(React Native) 동시 타깃
- 스키마가 자주 바뀌는 사용자/콘텐츠 서비스
- 지리/텍스트 검색, 피드/대시보드(집계 파이프라인)

과제 개요

MERN Stack을 이용한 Ajou Campus FoodMap Web Service

학습 목표

- Database 기본 개념 이해
- Database를 연동하여 백엔드 REST API 개발
- 웹서비스 운영 환경 구축
- MERN Stack 개발 환경 전반 이해

필수 구현 사항

- Express + MongoDB(Mongoose)로 백엔드 REST API를 구현
- React 프론트엔드와 연동
- Admin 운영 환경 개발
- 제보(Submission) → 승인 시 실제 맛집(Restaurant)으로 등록

마감일

- 10월 12일 11시 59분까지(2주간)

GitHub URL

- (예: https://github.com/***username***/pwd-week5)

Render 서버 URL(MongoDB Atlas 연동)

- (예: https://pwd-week5-**[username]**.onrender.com)

React App URL(Render 서버 연동)

- (예: https://pwd-week3-**[username]**.netlify.app)
- (예: https://pwd-week3-**[username]**.vercel.app)

Q&A